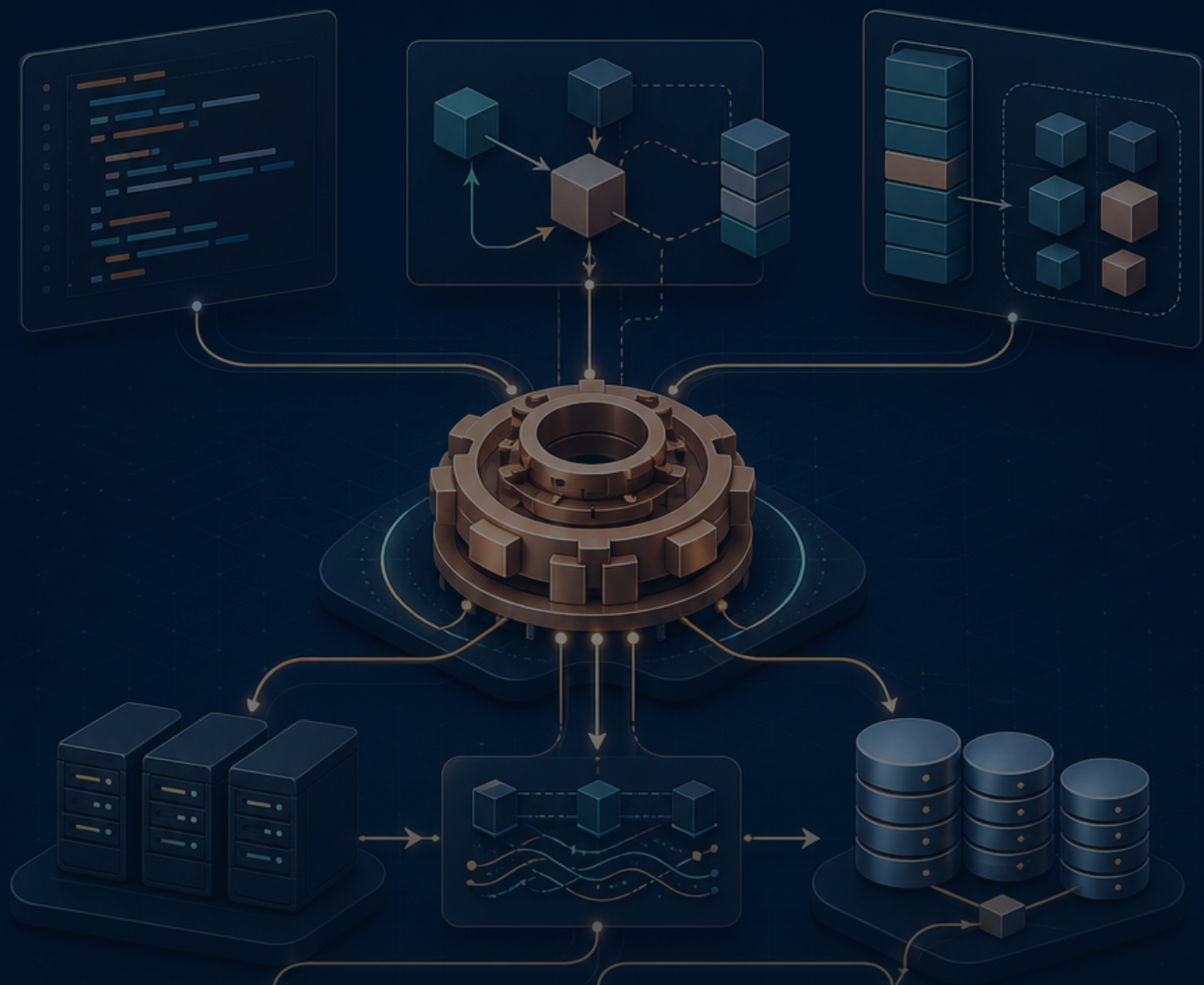




VOLUME 00 / RUST BACKEND MASTERY

学習ガイド — コンピュータとWebから完成像まで

プログラム未経験者がRustバックエンド開発の全体像と安全な学び方をつかむ導入巻

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

用語を先に暗記せず、入力サーバーとDBを通して画面へ戻る一周を見てから、各技術の役割を配置します。

到達目標

- ✓ 端末・ファイル・プロセス・ポートを区別できる
- ✓ ブラウザからDBまでの経路を説明できる
- ✓ コンパイラの指摘を学習材料として読める
- ✓ 14巻の学習順と完成条件を自分で管理できる

学習地図

01 コンピュータ・OS・アプリ

コンピュータを『CPUが命令を実行し、メモリ上の値を読み書きし、OSが資源を仲介する装置』として捉えます。Rustプログラムも特別なものではなく、OSが起動する一つのプロセスです。

02 ファイル・フォルダ・パス

ソースコード、設定、DB、ビルド成果物はすべてファイルです。相対パスは『今いるフォルダ』を基準に変わるため、初心者環境構築エラーの多くは現在位置の取り違いから生まれます。

03 端末・シェル・コマンド

端末は文字でOSへ依頼する窓、シェルはコマンドを解釈するプログラムです。PowerShellとBashでは環境変数やパスの書き方が異なるため、教材のコマンドがどちら向けかを確認します。

04 ソースコードから実行まで

人が書いたソースはrustcが機械語へ変換し、Cargoが依存関係とビルド手順を管理します。コンパイル時の失敗と、起動後の実行時失敗を区別します。

05 ブラウザ・URL・HTTP

ブラウザへURLを入力すると、名前解決、接続、HTTPリクエスト、サーバー処理、レスポンス、HTML描画が順に起きます。Webアプリの不具合はこの境界ごとに切り分けます。

06 サーバー・ポート・localhost

localhostは自分のPCを指す名前、ポートは同じPC上のどのプロセスへ届けるかを表す番号です。0.0.0.0へのbindは外部インターフェースでも待ち受ける意味で、公開URLそのものではありません。

07 データベースと永続化

変数の値はプロセス終了で消えます。DBは再起動後も残すデータを、表・行・列・制約・トランザクションとして管理します。TursoはlibSQL互換DBをネットワーク越しに利用できます。

08 認証と認可

認証は『誰か』を確かめ、認可は『何をしてよいか』を判断します。ログイン済みでも他人のノートを編集できてはいけません。パスワードは復号可能な形で保存しません。

09 コンパイラエラーとの付き合い方

Rustのエラーは失敗宣告ではなく、コンパイラが見つけた契約違反の診断書です。エラーコード、原因行、expected/found、helpを順に読み、最小変更で再コンパイルします。

10 学習サイクルと完成像

教材を読んだ量ではなく、自分で予想し、実行し、説明し、壊し、直し、テストへ固定できたかで進捗を測ります。最終成果はRustNoteをFly.ioで公開し、認証付きCRUDを運用できることです。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

CPUとメモリの役割

STEP 2

OSとプロセスの境界

STEP 3

アプリとデータの区別

STEP 4

終了コードと標準出力

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

コンピュータ・OS・アプリ

UNIT 01・CONCEPT

コンピュータ・OS・アプリ

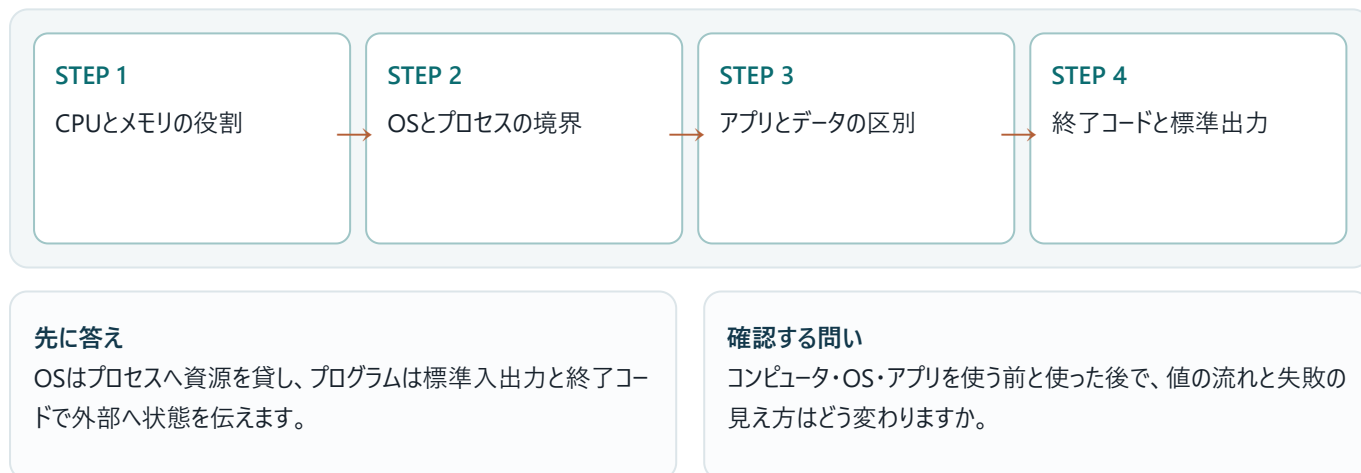
コンピュータを『CPUが命令を実行し、メモリ上の値を読み書きし、OSが資源を仲介する装置』として捉えます。Rustプログラムも特別なものではなく、OSが起動する一つのプロセスです。

この単元の到達点

- CPUとメモリの役割
- OSとプロセスの境界
- アプリとデータの区別
- 終了コードと標準出力

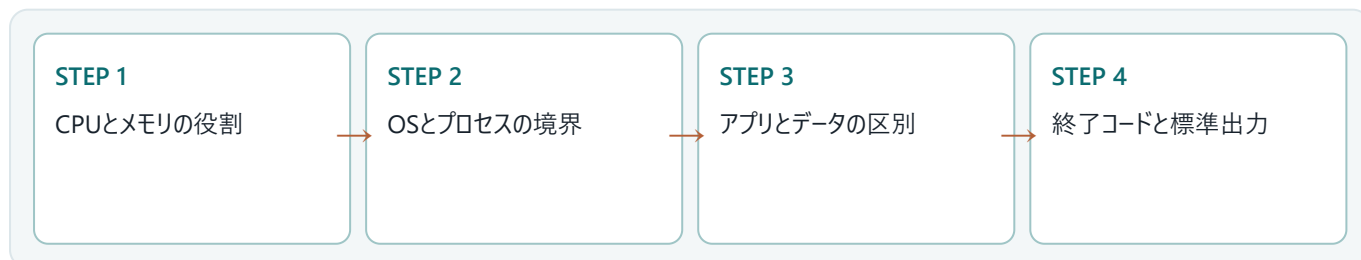
なぜバックエンドで必要か

サーバーは長時間動くプロセスです。OSからポート、メモリ、ファイル、環境変数を借り、終了時には接続を安全に閉じます。



UNIT 01・MENTAL MODEL

コンピュータ・OS・アプリを図でつかむ



コンピュータを『CPUが命令を実行し、メモリ上の値を読み書きし、OSが資源を仲介する装置』として捉えます。Rustプログラムも特別なものではなく、OSが起動する一つのプロセスです。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	CPUとメモリの役割	CPUとメモリの役割 — 入力と前提を確認し、境界を曖昧にしない。
2	OSとプロセスの境界	OSとプロセスの境界 — 値がどの順で変換されるかを追跡する。
3	アプリとデータの区別	アプリとデータの区別 — 失敗を再現し、型またはログから原因を特定する。
4	終了コードと標準出力	終了コードと標準出力 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

コンピュータ・OS・アプリとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

コンピュータ・OS・アプリだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn main() {
    println!("Rust process started");
    std::process::exit(0);
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>println!("Rust process started");</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>std::process::exit(0);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がCPUとメモリの役割の条件を満たすか確認する。
- ✓ OSとプロセスの境界を実行し、途中の型と状態を追う。
- ✓ アプリとデータの区別で失敗を成功経路から分離する。
- ✓ 終了コードと標準出力により、出力と副作用を検証する。

型を声に出す コンピュータ・OS・アプリに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
Rust process started
process exited with code 0
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn main() {
    println!("pid={}", std::process::id());
    let ok = true;
    std::process::exit(if ok { 0 } else { 1 });
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01 · BACKEND APPLICATION

バックエンドのどこで使うか

サーバーは長時間動くプロセスです。OSからポート、メモリ、ファイル、環境変数を借り、終了時には接続を安全に閉じます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	CPUとメモリの役割に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	OSとプロセスの境界の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	アプリとデータの区別または終了コードと標準出力の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

コンピュータ・OS・アプリを再現して切り分ける

コンピュータ・OS・アプリの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: CPUとメモリの役割
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: OSとプロセスの境界
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: アプリとデータの区別
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 終了コードと標準出力

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	CPUとメモリの役割	CPUとメモリの役割 — 入力と前提を確認し、境界を曖昧にしない。
2	OSとプロセスの境界	OSとプロセスの境界 — 値がどの順で変換されるかを追跡する。
3	アプリとデータの区別	アプリとデータの区別 — 失敗を再現し、型またはログから原因を特定する。
4	終了コードと標準出力	終了コードと標準出力 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 OSはプロセスへ資源を貸し、プログラムは標準入出力と終了コードで外部へ状態を伝えます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

現在動いているRustプログラムのプロセスIDを表示し、終了コード0と1を試してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn main() {  
    println!("pid={}", std::process::id());  
    let ok = true;  
    std::process::exit(if ok { 0 } else { 1 });  
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ OSはプロセスへ資源を貸し、プログラムは標準入出力と終了コードで外部へ状態を伝えます。
- ✓ コンピュータ・OS・アプリとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

ファイル・フォルダ・パス

UNIT 02 · CONCEPT

ファイル・フォルダ・パス

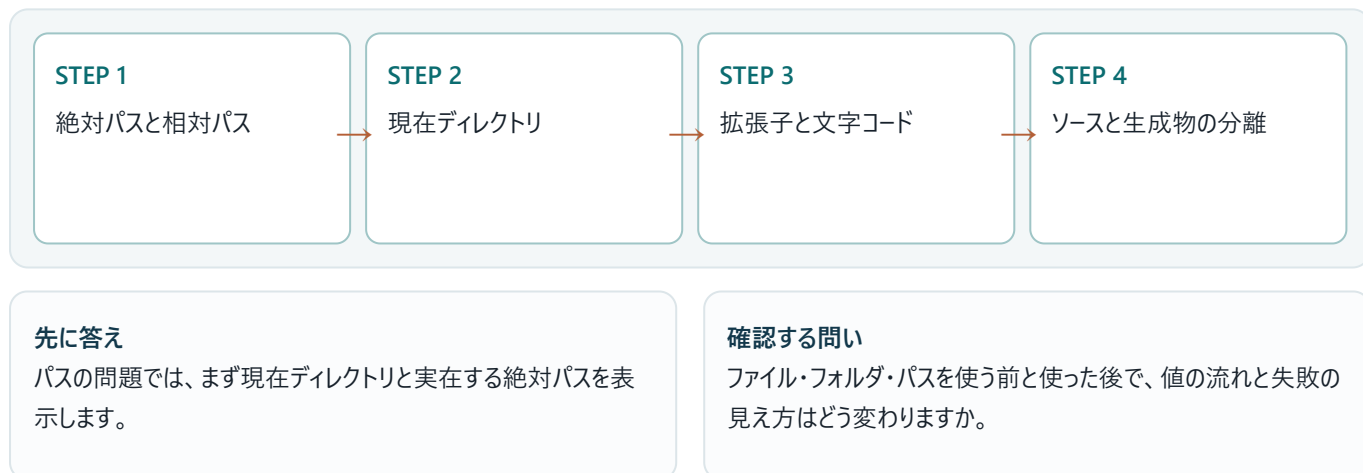
ソースコード、設定、DB、ビルド成果物はすべてファイルです。相対パスは『今いるフォルダ』を基準に変わるため、初心者の環境構築エラーの多くは現在位置の取り違えから生まれます。

この単元の到達点

- 絶対パスと相対パス
- 現在ディレクトリ
- 拡張子と文字コード
- ソースと生成物の分離

なぜバックエンドで必要か

サーバーは設定や静的ファイルを読みます。実行場所に依存する相対パスは開発PCとコンテナで異なるため、基準位置を明示します。



UNIT 02 · MENTAL MODEL

ファイル・フォルダ・パスを図でつかむ



ソースコード、設定、DB、ビルド成果物はすべてファイルです。相対パスは『今いるフォルダ』を基準に変わるため、初心者の環境構築エラーの多くは現在位置の取り違えから生まれます。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	絶対パスと相対パス	絶対パスと相対パス — 入力と前提を確認し、境界を曖昧にしない。
2	現在ディレクトリ	現在ディレクトリ — 値がどの順で変換されるかを追跡する。
3	拡張子と文字コード	拡張子と文字コード — 失敗を再現し、型またはログから原因を特定する。
4	ソースと生成物の分離	ソースと生成物の分離 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

ファイル・フォルダ・パスとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE まず動く最小コード

ファイル・フォルダ・パスだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::env;
fn main() -> std::io::Result<()> {
    let here = env::current_dir()?;
    println!("cwd={}", here.display());
    Ok(())
}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::env;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() -> std::io::Result<> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let here = env::current_dir()?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>println!("{}", here.display());</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>Ok()</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が絶対パスと相対パスの条件を満たすか確認する。
- ✓ 現在ディレクトリを実行し、途中の型と状態を追う。
- ✓ 拡張子と文字コードで失敗を成功経路から分離する。
- ✓ ソースと生成物の分離により、出力と副作用を検証する。

型を声に出す ファイル・フォルダ・パスに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

`cwd=C:\Users\you\projects\rust-lab`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::{fs, path::PathBuf};
fn main() -> std::io::Result<()> {
    let path = PathBuf::from("assets").join("message.txt");
    println!("{}", fs::read_to_string(&path)?);
    Ok(())
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

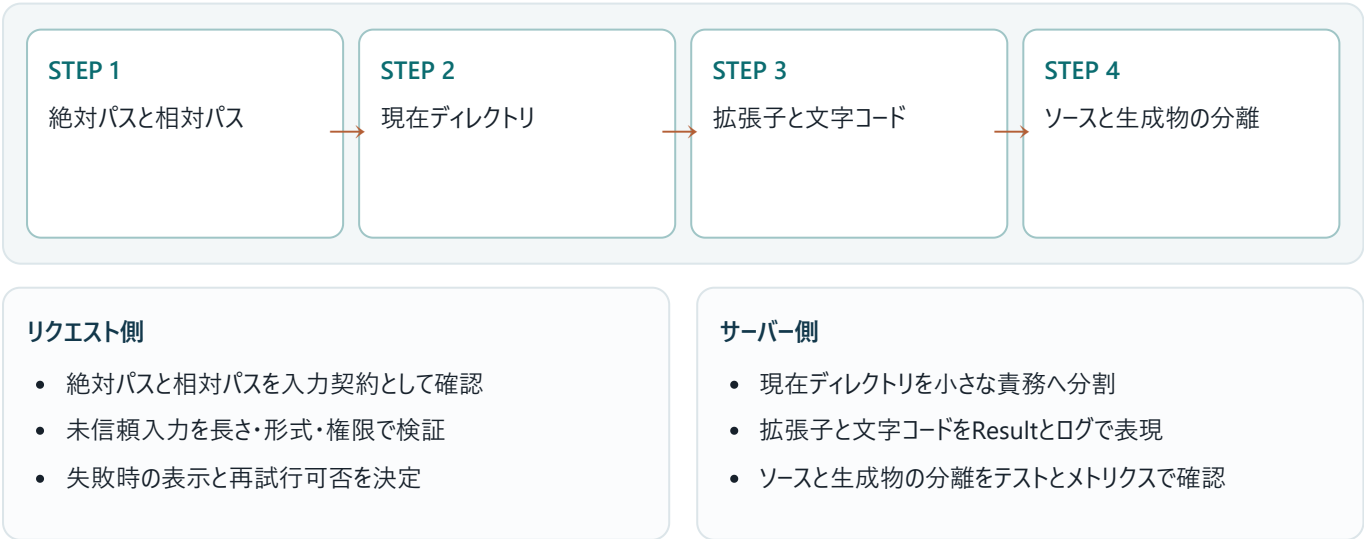
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

サーバーは設定や静的ファイルを読みます。実行場所に依存する相対パスは開発PCとコンテナで異なるため、基準位置を明示します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	絶対パスと相対パスに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	現在ディレクトリの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	拡張子と文字コードまたはソースと生成物の分離の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

ファイル・フォルダ・パスを再現して切り分ける

ファイル・フォルダ・パスの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 絶対パスと相対パス
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 現在ディレクトリ
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 拡張子と文字コード
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: ソースと生成物の分離

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	絶対パスと相対パス	絶対パスと相対パス — 入力と前提を確認し、境界を曖昧にしない。
2	現在ディレクトリ	現在ディレクトリ — 値がどの順で変換されるかを追跡する。
3	拡張子と文字コード	拡張子と文字コード — 失敗を再現し、型またはログから原因を特定する。
4	ソースと生成物の分離	ソースと生成物の分離 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 パスの問題では、まず現在ディレクトリと実在する絶対パスを表示します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

assets/message.txtを作り、相対パスで読み込み、エラー時には試したパスも表示してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::{fs, path::PathBuf};
fn main() -> std::io::Result<()> {
    let path = PathBuf::from("assets").join("message.txt");
    println!("{}", fs::read_to_string(&path)?);
    Ok(())
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ パスの問題では、まず現在ディレクトリと実在する絶対パスを表示します。
- ✓ ファイル・フォルダ・パスとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

端末・シェル・コマンド

UNIT 03 · CONCEPT

端末・シェル・コマンド

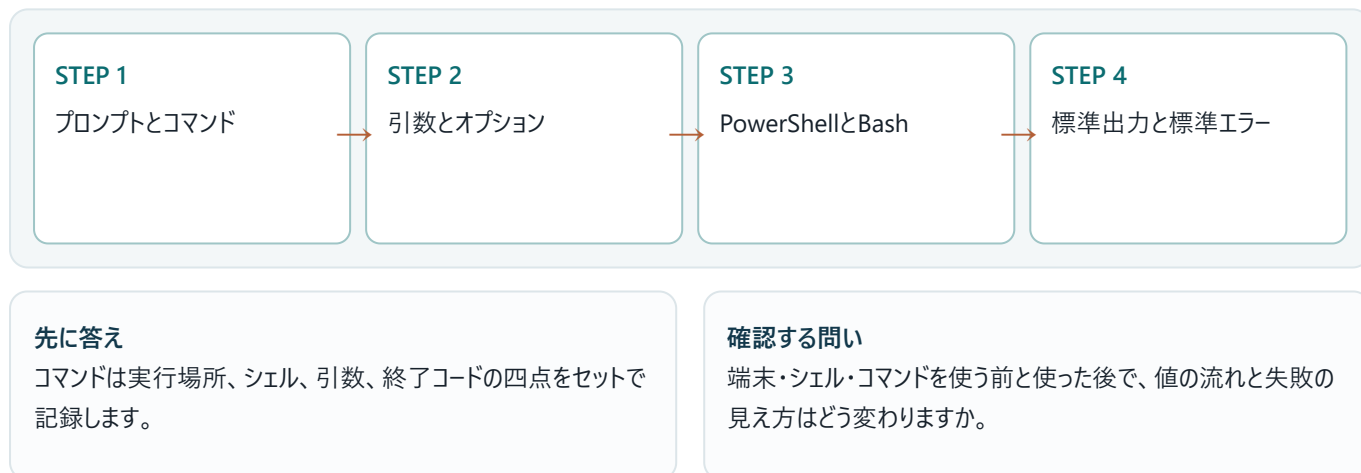
端末は文字でOSへ依頼する窓、シェルはコマンドを解釈するプログラムです。PowerShellとBashでは環境変数やパスの書き方が異なるため、教材のコマンドがどちら向けかを確認します。

この単元の到達点

- プロンプトとコマンド
- 引数とオプション
- PowerShellとBash
- 標準出力と標準エラー

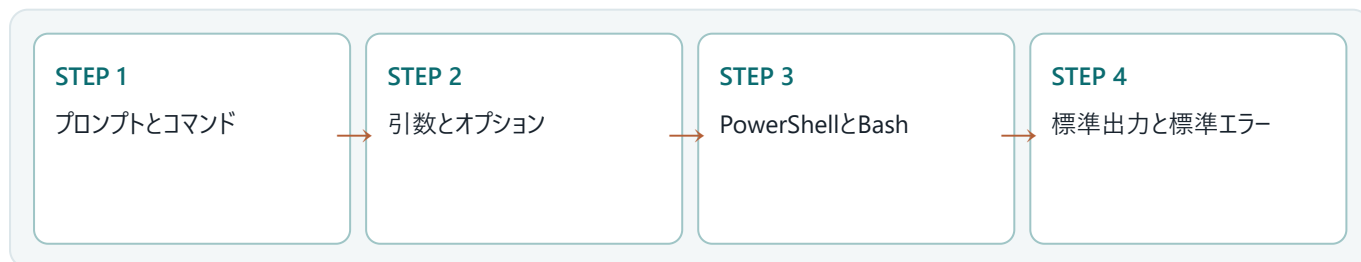
なぜバックエンドで必要か

ビルド、テスト、DB操作、デプロイはコマンドから再現できる形で残します。GUIだけの操作は共有と自動化が難しくなります。



UNIT 03 · MENTAL MODEL

端末・シェル・コマンドを図でつかむ



端末は文字でOSへ依頼する窓、シェルはコマンドを解釈するプログラムです。PowerShellとBashでは環境変数やパスの書き方が異なるため、教材のコマンドがどちら向けかを確認します。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	プロンプトとコマンド	プロンプトとコマンド — 入力と前提を確認し、境界を曖昧にしない。
2	引数とオプション	引数とオプション — 値がどの順で変換されるかを追跡する。
3	PowerShellとBash	PowerShellとBash — 失敗を再現し、型またはログから原因を特定する。
4	標準出力と標準エラー	標準出力と標準エラー — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

端末・シェル・コマンドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE まず動く最小コード

端末・シェル・コマンドだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
$PSVersionTable.PSVersion
Get-Location
Get-ChildItem
$env:RUST_LOG='debug'
```

● ● ● 実行コマンド

PowerShellで上から一行ずつ実行

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
\$PSVersionTable.PSVersion	この行が受け取る値、作る値、副作用の有無を確認します。
Get-Location	この行が受け取る値、作る値、副作用の有無を確認します。
Get-ChildItem	この行が受け取る値、作る値、副作用の有無を確認します。
\$env:RUST_LOG='debug'	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力プロンプトとコマンドの条件を満たすか確認する。
- ✓ 引数とオプションを実行し、途中の型と状態を追う。
- ✓ PowerShellとBashで失敗を成功経路から分離する。
- ✓ 標準出力と標準エラーにより、出力と副作用を検証する。

型を声に出す 端末・シェル・コマンドに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

```

● ● ● 実行結果

Major  Minor  Patch
7       5      2

Path
C:\projects\rust-lab

```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
# PowerShell
$env:APP_MODE='development'
Get-ChildItem
$LASTEXITCODE

# Bash
export APP_MODE=development
ls
echo $?
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

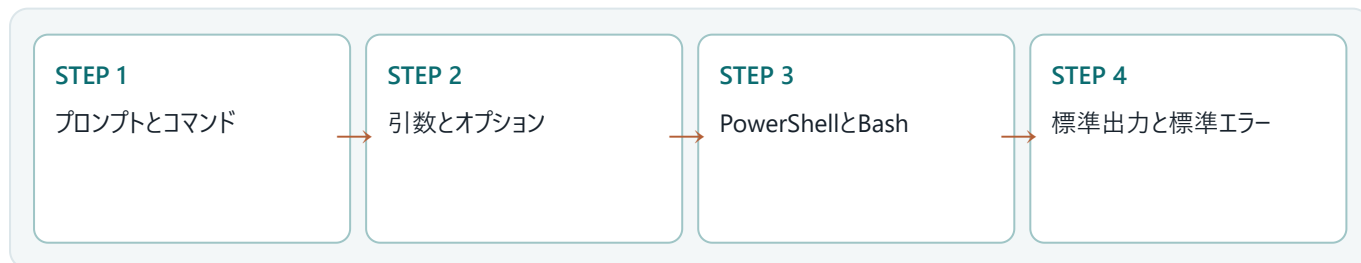
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

ビルド、テスト、DB操作、デプロイはコマンドから再現できる形で残します。GUIだけの操作は共有と自動化が難しくなります。



リクエスト側

- プロンプトとコマンドを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 引数とオプションを小さな責務へ分割
- PowerShellとBashをResultとログで表現
- 標準出力と標準エラーをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	プロンプトとコマンドに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違ふ	引数とオプションの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	PowerShellとBashまたは標準出力と標準エラーの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

端末・シェル・コマンドを再現して切り分ける

端末・シェル・コマンドの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: プロンプトとコマンド
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 引数とオプション
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: PowerShellとBash
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 標準出力と標準エラー

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	プロンプトとコマンド	プロンプトとコマンド — 入力と前提を確認し、境界を曖昧にしない。
2	引数とオプション	引数とオプション — 値がどの順で変換されるかを追跡する。
3	PowerShellとBash	PowerShellとBash — 失敗を再現し、型またはログから原因を特定する。
4	標準出力と標準エラー	標準出力と標準エラー — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 コマンドは実行場所、シェル、引数、終了コードの四点をセットで記録します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

同じフォルダー一覧、環境変数設定、終了コード確認をPowerShellとBashの両方で書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
# PowerShell
$env:APP_MODE='development'
Get-ChildItem
$LASTEXITCODE

# Bash
export APP_MODE=development
ls
echo $?
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ コマンドは実行場所、シェル、引数、終了コードの四点をセットで記録します。
- ✓ 端末・シェル・コマンドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

ソースコードから実行まで

UNIT 04 · CONCEPT

ソースコードから実行まで

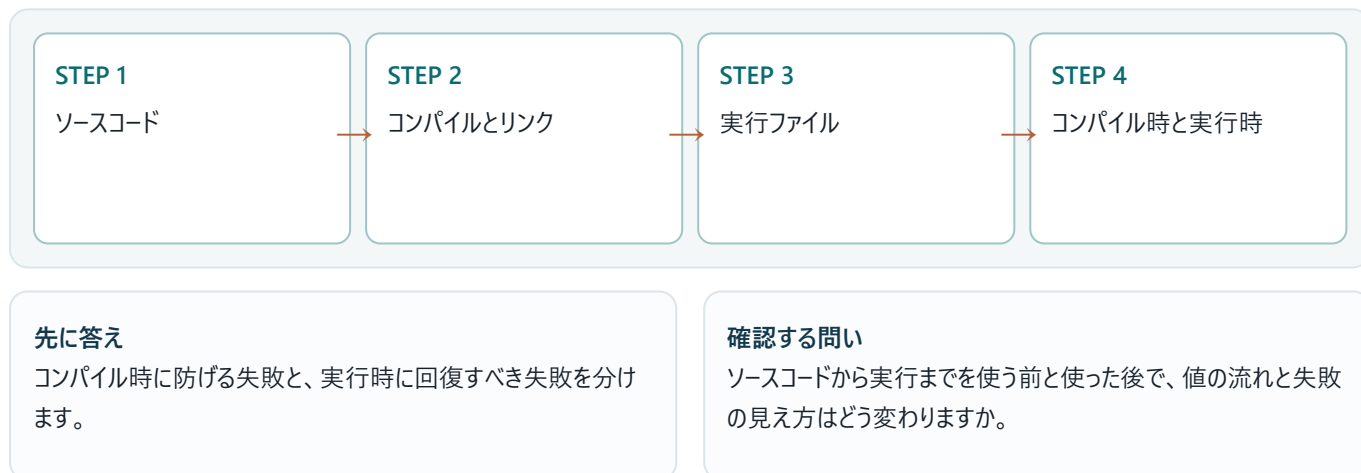
人が書いたソースはrustcが機械語へ変換し、Cargoが依存関係とビルド手順を管理します。コンパイル時の失敗と、起動後の実行時失敗を区別します。

この単元の到達点

- ソースコード
- コンパイルとリンク
- 実行ファイル
- コンパイル時と実行時

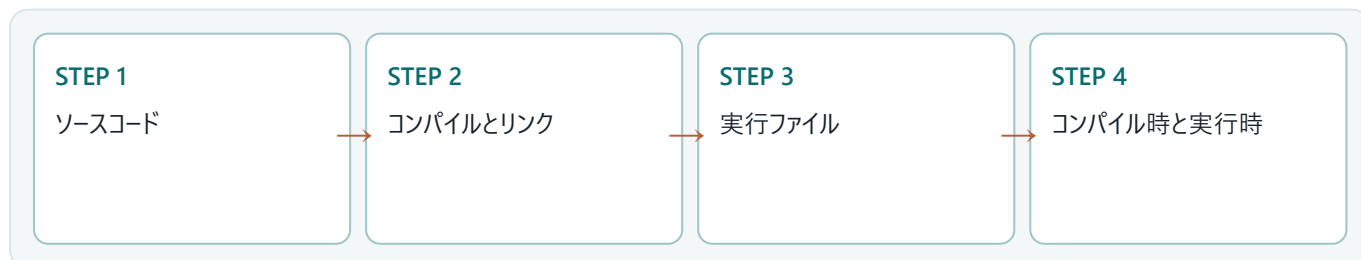
なぜバックエンドで必要か

Rustバックエンドは起動前に多くの型不整合を排除できます。一方、ネットワーク切断やDB障害は実行時の失敗としてResultで扱います。



UNIT 04 · MENTAL MODEL

ソースコードから実行までを図でつかむ



人が書いたソースはrustcが機械語へ変換し、Cargoが依存関係とビルド手順を管理します。コンパイル時の失敗と、起動後の実行時失敗を区別します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	ソースコード	ソースコード — 入力と前提を確認し、境界を曖昧にしない。
2	コンパイルとリンク	コンパイルとリンク — 値がどの順で変換されるかを追跡する。
3	実行ファイル	実行ファイル — 失敗を再現し、型またはログから原因を特定する。
4	コンパイル時と実行時	コンパイル時と実行時 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

ソースコードから実行までとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

ソースコードから実行までだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn double(value: i32) -> i32 { value * 2 }
fn main() {
    let result = double(21);
    println!("result={result}");
}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn double(value: i32) -> i32 { value * 2 }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let result = double(21);</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>println!("{}", result);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力ソースコードの条件を満たすか確認する。
- ✓ コンパイルとリンクを実行し、途中の型と状態を追う。
- ✓ 実行ファイルで失敗を成功経路から分離する。
- ✓ コンパイル時と実行時により、出力と副作用を検証する。

型を声に出す ソースコードから実行までに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

result=42

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn double(value: i32) -> Result<i32, &'static str> {
    value.checked_mul(2).ok_or("overflow")
}
fn main() -> Result<(), &'static str> {
    println!("{}", double(21)?);
    Ok(())
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

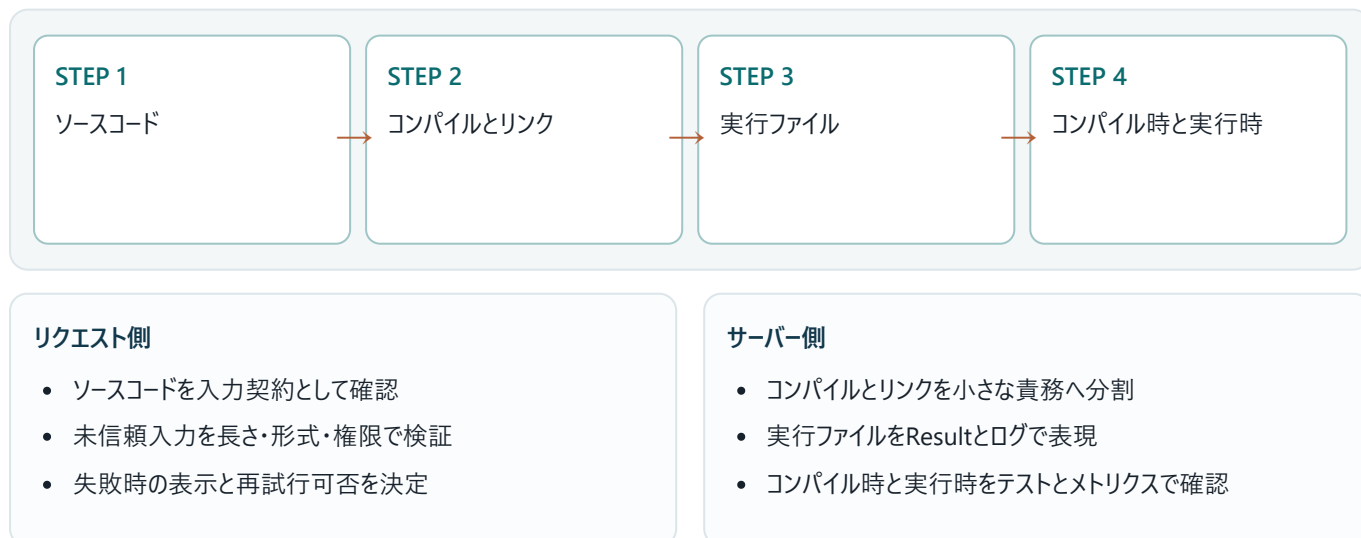
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

Rustバックエンドは起動前に多くの型不整合を排除できます。一方、ネットワーク切断やDB障害は実行時の失敗としてResultで扱います。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	ソースコードに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	コンパイルとリンクの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	実行ファイルまたはコンパイル時と実行時の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

ソースコードから実行までを再現して切り分ける

ソースコードから実行までの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: ソースコード
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: コンパイルとリンク
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 実行ファイル
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: コンパイル時と実行時

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	ソースコード	ソースコード — 入力と前提を確認し、境界を曖昧にしない。
2	コンパイルとリンク	コンパイルとリンク — 値がどの順で変換されるかを追跡する。
3	実行ファイル	実行ファイル — 失敗を再現し、型またはログから原因を特定する。
4	コンパイル時と実行時	コンパイル時と実行時 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 コンパイル時に防げる失敗と、実行時に回復すべき失敗を分けます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

引数の型を*i32*から*String*へ変えてコンパイルエラーを読み、次に*checked_mul*でオーバーフローを扱ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn double(value: i32) -> Result<i32, &'static str> {  
    value.checked_mul(2).ok_or("overflow")  
}  
  
fn main() -> Result<(), &'static str> {  
    println!("{}", double(21)?);  
    Ok(())  
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ コンパイル時に防げる失敗と、実行時に回復すべき失敗を分けます。
- ✓ ソースコードから実行までとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

ブラウザ・URL・HTTP

UNIT 05 · CONCEPT

ブラウザ・URL・HTTP

ブラウザへURLを入力すると、名前解決、接続、HTTPリクエスト、サーバー処理、レスポンス、HTML描画が順に起きます。Webアプリの不具合はこの境界ごとに切り分けます。

この単元の到達点

- URLの構造
- DNSと接続
- HTTP要求と応答
- HTML描画

なぜバックエンドで必要か

HTTPサーバーはポートで待ち受け、リクエストを検証し、状態コード・ヘッダー・本文を返します。Topcoatが抽象化しても基礎は障害切り分けに必要です。



UNIT 05 · MENTAL MODEL

ブラウザ・URL・HTTPを図でつかむ



ブラウザへURLを入力すると、名前解決、接続、HTTPリクエスト、サーバー処理、レスポンス、HTML描画が順に起きます。Webアプリの不具合はこの境界ごとに切り分けます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認

認めます。

順序	観察対象	初心者が確認すること
1	URLの構造	URLの構造 — 入力と前提を確認し、境界を曖昧にしない。
2	DNSと接続	DNSと接続 — 値がどの順で変換されるかを追跡する。
3	HTTP要求と応答	HTTP要求と応答 — 失敗を再現し、型またはログから原因を特定する。
4	HTML描画	HTML描画 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

ブラウザ・URL・HTTPとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05 · MINIMUM CODE

まず動く最小コード

ブラウザ・URL・HTTPだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
use std::net::TcpListener;
fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:4000"?);
    println!("listening on http://127.0.0.1:4000");
    drop(listener);
    Ok(())
}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::net::TcpListener;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() -> std::io::Result<> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let listener = TcpListener::bind("127.0.0.1:4000");</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>println!("listening on http://127.0.0.1:4000");</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>drop(listener);</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力URLの構造の条件を満たすか確認する。
- ✓ DNSと接続を実行し、途中の型と状態を追う。
- ✓ HTTP要求と応答で失敗を成功経路から分離する。
- ✓ HTML描画により、出力と副作用を検証する。

型を声に出す ブラウザ・URL・HTTPに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

ブラウザ・URL・HTTP

URLを入力すると、サーバーから返されたHTMLが見出しとカードとして描画されます。

観察ポイント

- URLの構造
- DNSと接続
- HTTP要求と応答

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn main() {
    let url = "https://example.com:443/notes?page=2";
    println!("scheme=https host=example.com port=443 path=/notes query=page=2");
    assert!(url.starts_with("https://"));
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 · BACKEND APPLICATION

バックエンドのどこで使うか

HTTPサーバーはポートで待ち受け、リクエストを検証し、状態コード・ヘッダー・本文を返します。Topcoatが抽象化しても基礎は障害切り分けに必要です。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	URLの構造に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	DNSと接続の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	HTTP要求と応答またはHTML描画の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

ブラウザ・URL・HTTPを再現して切り分ける

ブラウザ・URL・HTTPの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: URLの構造
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: DNSと接続
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: HTTP要求と応答
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: HTML描画

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	URLの構造	URLの構造 — 入力と前提を確認し、境界を曖昧にしない。
2	DNSと接続	DNSと接続 — 値がどの順で変換されるかを追跡する。
3	HTTP要求と応答	HTTP要求と応答 — 失敗を再現し、型またはログから原因を特定する。
4	HTML描画	HTML描画 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 Webの一周は、URL→接続→HTTP→サーバー→DB→HTTP→描画です。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

URL `https://example.com:443/notes?page=2` をscheme、host、port、path、queryへ分解してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn main() {  
    let url = "https://example.com:443/notes?page=2";  
    println!("scheme=https host=example.com port=443 path=/notes query?page=2");  
    assert!(url.starts_with("https://"));  
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ Webの一周は、URL→接続→HTTP→サーバー→DB→HTTP→描画です。
- ✓ ブラウザ・URL・HTTPとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

サーバー・ポート・localhost

UNIT 06 · CONCEPT

サーバー・ポート・localhost

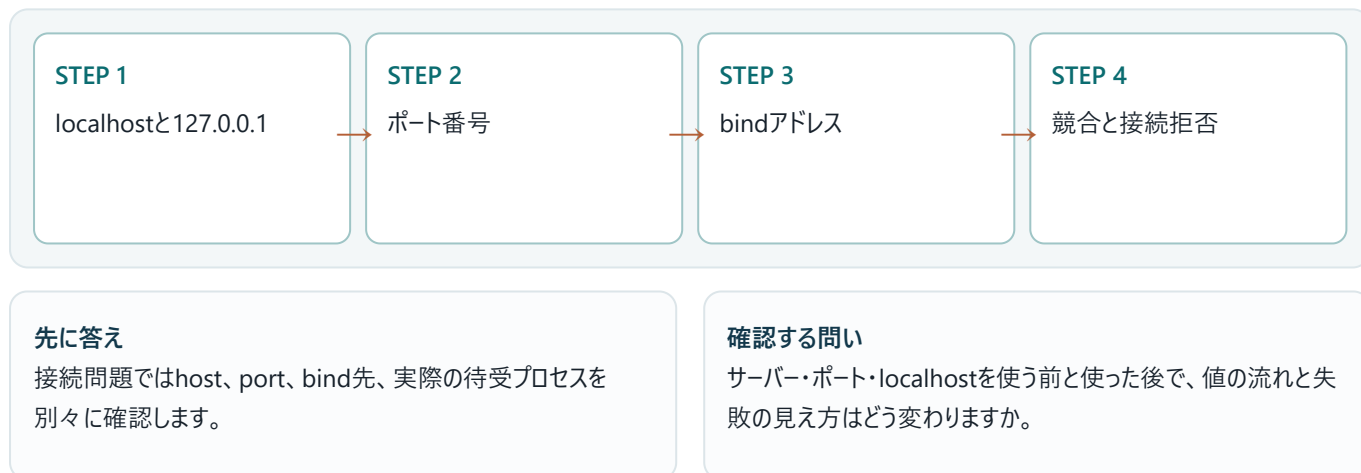
localhostは自分のPCを指す名前、ポートは同じPC上のどのプロセスへ届けるかを表す番号です。0.0.0.0へのbindは外部インターフェースでも待ち受ける意味で、公開URLそのものではありません。

この単元の到達点

- localhostと127.0.0.1
- ポート番号
- bindアドレス
- 競合と接続拒否

なぜバックエンドで必要か

Fly.ioのコンテナでは0.0.0.0とPORT=8080で待ち受けます。127.0.0.1だけにbindするとプロキシから到達できません。



UNIT 06 · MENTAL MODEL

サーバー・ポート・localhostを図でつかむ



localhostは自分のPCを指す名前、ポートは同じPC上のどのプロセスへ届けるかを表す番号です。0.0.0.0へのbindは外部インターフェースでも待ち受ける意味で、公開URLそのものではありません。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	localhostと127.0.0.1	localhostと127.0.0.1 — 入力と前提を確認し、境界を曖昧にしない。
2	ポート番号	ポート番号 — 値がどの順で変換されるかを追跡する。
3	bindアドレス	bindアドレス — 失敗を再現し、型またはログから原因を特定する。
4	競合と接続拒否	競合と接続拒否 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

サーバー・ポート・localhostとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06 · MINIMUM CODE まず動く最小コード

サーバー・ポート・localhostだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::net::TcpListener;
fn main() {
    match TcpListener::bind("127.0.0.1:3000") {
        Ok(_) => println!("port 3000 is available"),
        Err(e) => eprintln!("cannot bind: {e}"),
    }
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::net::TcpListener;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>match TcpListener::bind("127.0.0.1:3000") {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>Ok(_) => println!("port 3000 is available"),</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>Err(e) => eprintln!("cannot bind: {e}"),</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がlocalhostと127.0.0.1の条件を満たすか確認する。
- ✓ ポート番号を実行し、途中の型と状態を追う。
- ✓ bindアドレスで失敗を成功経路から分離する。
- ✓ 競合と接続拒否により、出力と副作用を検証する。

型を声に出す サーバー・ポート・localhostに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

port 3000 is available

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::net::TcpListener;
fn main() -> std::io::Result<()> {
    let host = std::env::var("HOST").unwrap_or_else(|_| "127.0.0.1".into());
    let port = std::env::var("PORT").unwrap_or_else(|_| "3000".into());
    let _listener = TcpListener::bind(format!("{host}:{port}"))?;
    println!("listening on {host}:{port}");
    Ok(())
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

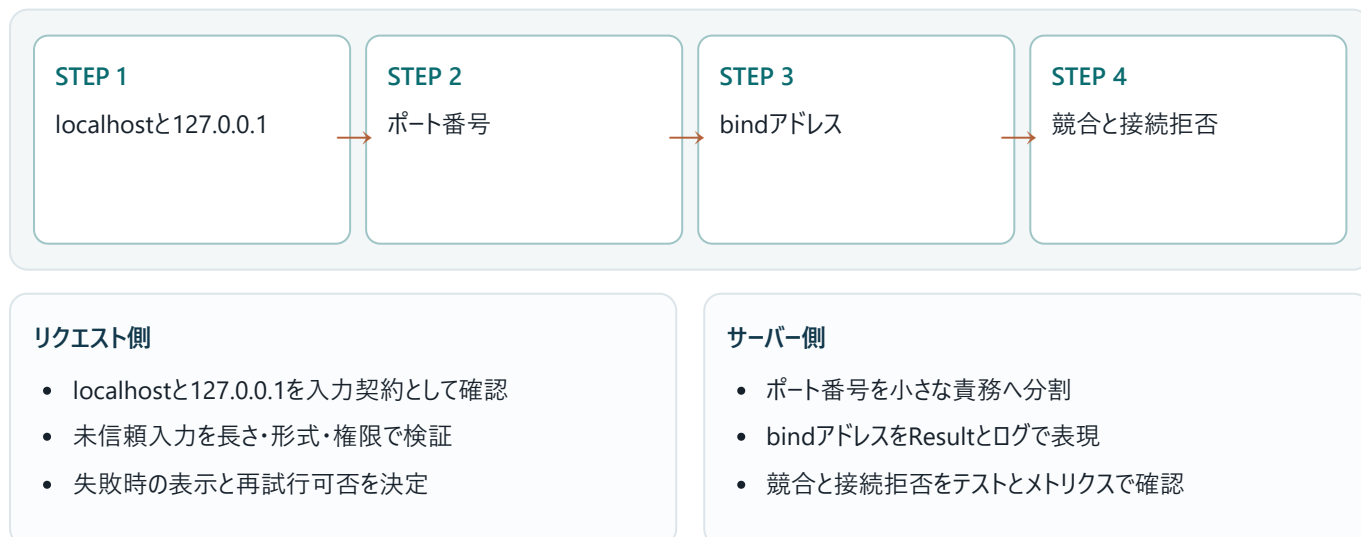
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

Fly.ioのコンテナでは0.0.0.0とPORT=8080で待ち受けます。127.0.0.1だけにbindするとプロキシから到達できません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	localhostと127.0.0.1に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	ポート番号の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	bindアドレスまたは競合と接続拒否の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

サーバー・ポート・localhostを再現して切り分ける

サーバー・ポート・localhostの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: localhostと127.0.0.1
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: ポート番号
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: bindアドレス
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 競合と接続拒否

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	localhostと127.0.0.1	localhostと127.0.0.1 — 入力と前提を確認し、境界を曖昧にしない。
2	ポート番号	ポート番号 — 値がどの順で変換されるかを追跡する。
3	bindアドレス	bindアドレス — 失敗を再現し、型またはログから原因を特定する。
4	競合と接続拒否	競合と接続拒否 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 接続問題ではhost、port、bind先、実際の待受プロセスを別々に確認します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

同じプログラムを二つ起動してAddress already in useを再現し、使用中プロセスを調べてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::net::TcpListener;
fn main() -> std::io::Result<()> {
    let host = std::env::var("HOST").unwrap_or_else(|_| "127.0.0.1".into());
    let port = std::env::var("PORT").unwrap_or_else(|_| "3000".into());
    let _listener = TcpListener::bind(format!("{host}:{port}"))?;
    println!("listening on {host}:{port}");
    Ok(())
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 接続問題ではhost、port、bind先、実際の待受プロセスを別々に確認します。
- ✓ サーバー・ポート・localhostとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

データベースと永続化

UNIT 07 · CONCEPT

データベースと永続化

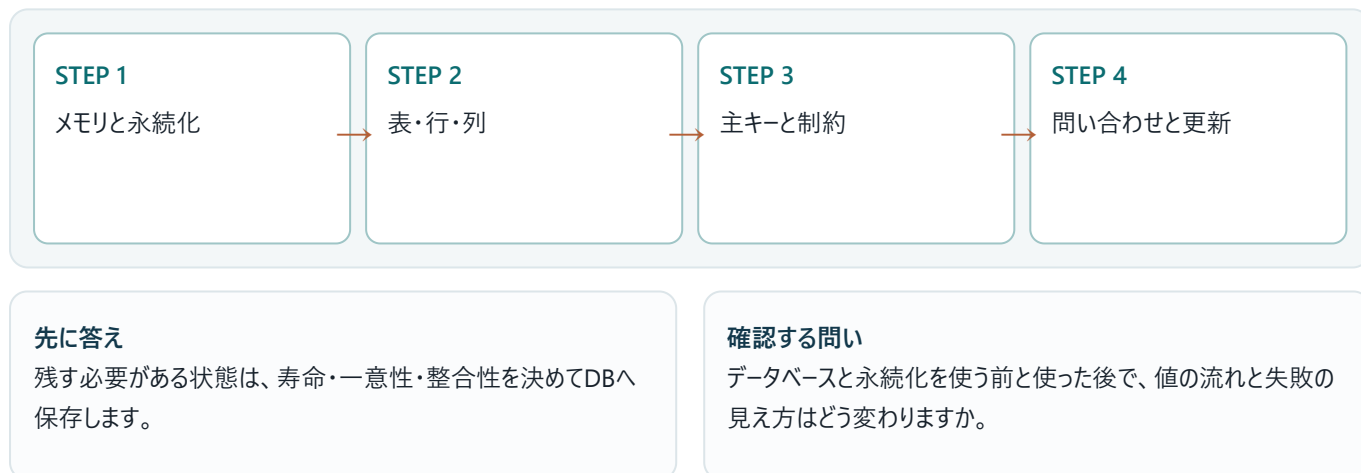
変数の値はプロセス終了で消えます。DBは再起動後も残すデータを、表・行・列・制約・トランザクションとして管理します。TursoはlibSQL互換DBをネットワーク越しに利用できます。

この単元の到達点

- メモリと永続化
- 表・行・列
- 主キーと制約
- 問い合わせと更新

なぜバックエンドで必要か

HTTPリクエストごとに作る一時変数と、利用者のノートやセッションのようにDBへ保存する状態を区別します。



UNIT 07 · MENTAL MODEL

データベースと永続化を図でつかむ



変数の値はプロセス終了で消えます。DBは再起動後も残すデータを、表・行・列・制約・トランザクションとして管理します。TursoはlibSQL互換DBをネットワーク越しに利用できます。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	メモリと永続化	メモリと永続化 — 入力と前提を確認し、境界を曖昧にしない。
2	表・行・列	表・行・列 — 値がどの順で変換されるかを追跡する。
3	主キーと制約	主キーと制約 — 失敗を再現し、型またはログから原因を特定する。
4	問い合わせと更新	問い合わせと更新 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

データベースと永続化とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE

まず動く最小コード

データベースと永続化だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn main() {
    let notes = vec!["ownership", "async"];
    for (id, title) in notes.iter().enumerate() {
        println!("id={} title={}", id + 1, title);
    }
}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let notes = vec!["ownership", "async"];</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>for (id, title) in notes.iter().enumerate() {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>println!("id={} title={}", id + 1, title);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がメモリと永続化の条件を満たすか確認する。
- ✓ 表・行・列を実行し、途中の型と状態を追う。
- ✓ 主キーと制約で失敗を成功経路から分離する。
- ✓ 問い合わせと更新により、出力と副作用を検証する。

型を声に出す データベースと永続化に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

id=1 title=ownership
id=2 title=async

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug)]
struct Note { id: u64, title: String }
fn main() {
    let mut notes = vec![Note { id: 1, title: "ownership".into() }];
    notes.retain(|n| n.id != 1);
    assert!(notes.is_empty());
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

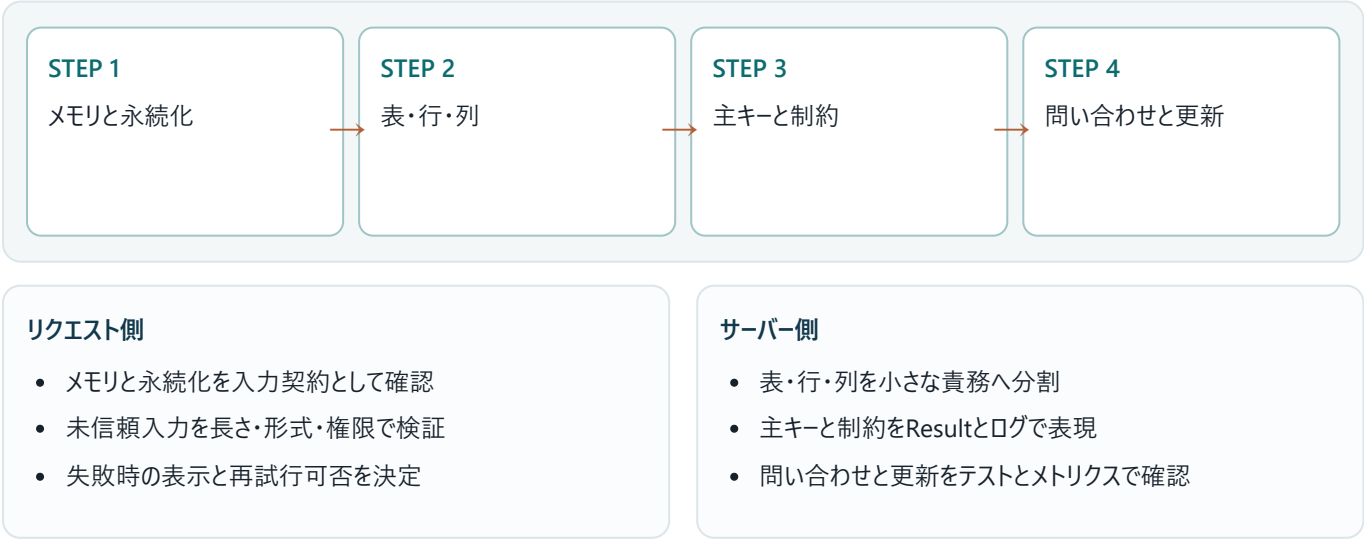
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

HTTPリクエストごとに作る一時変数と、利用者のノートやセッションのようにDBへ保存する状態を区別します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	メモリと永続化に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	表・行・列の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	主キーと制約または問い合わせと更新の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

データベースと永続化を再現して切り分ける

データベースと永続化の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: メモリと永続化
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 表・行・列
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 主キーと制約
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 問い合わせと更新

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	メモリと永続化	メモリと永続化 — 入力と前提を確認し、境界を曖昧にしない。
2	表・行・列	表・行・列 — 値がどの順で変換されるかを追跡する。
3	主キーと制約	主キーと制約 — 失敗を再現し、型またはログから原因を特定する。
4	問い合わせと更新	問い合わせと更新 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 残す必要がある状態は、寿命・一意性・整合性を決めてDBへ保存します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

メモリ上のノート一覧にcreate、list、deleteを実装し、プロセス終了で消えることを確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug)]
struct Note { id: u64, title: String }
fn main() {
    let mut notes = vec![Note { id: 1, title: "ownership".into() }];
    notes.retain(|n| n.id != 1);
    assert!(notes.is_empty());
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 残す必要がある状態は、寿命・一意性・整合性を決めてDBへ保存します。
- ✓ データベースと永続化とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

認証と認可

UNIT 08 · CONCEPT

認証と認可

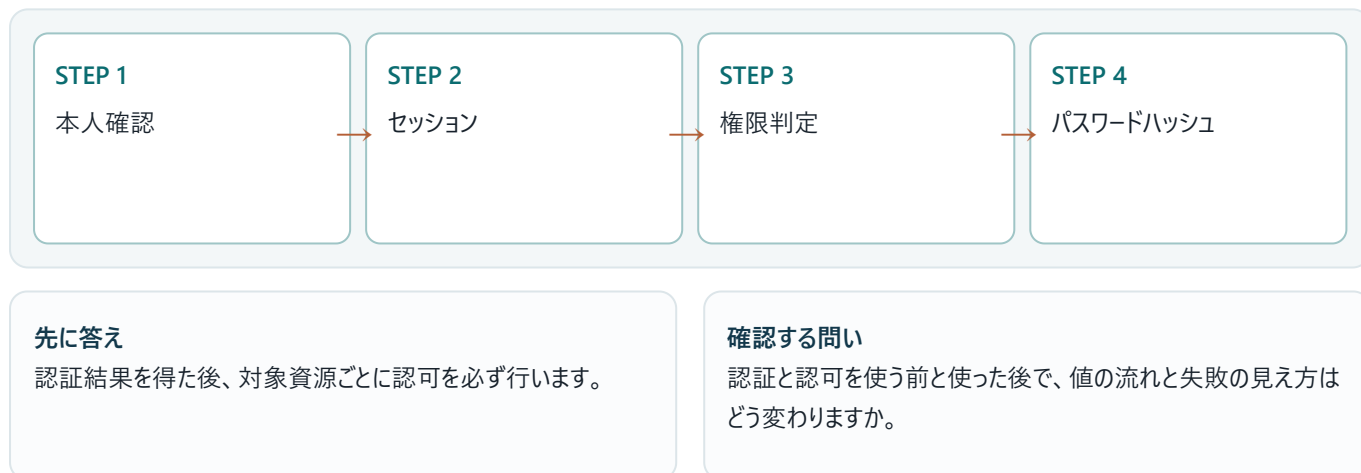
認証は『誰か』を確かめ、認可は『何をしてよいか』を判断します。ログイン済みでも他人のノートを編集できてはいけません。パスワードは復号可能な形で保存しません。

この単元の到達点

- 本人確認
- セッション
- 権限判定
- パスワードハッシュ

なぜバックエンドで必要か

各リクエストでセッションを検証し、得たuser_idをSQLのWHERE条件にも含めます。URLのIDだけを信用するとIDOR脆弱性になります。



UNIT 08 · MENTAL MODEL

認証と認可を図でつかむ



認証は『誰か』を確かめ、認可は『何をしてよいか』を判断します。ログイン済みでも他人のノートを編集できてはいけません。パスワードは復号可能な形で保存しません。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	本人確認	本人確認 — 入力と前提を確認し、境界を曖昧にしない。
2	セッション	セッション — 値がどの順で変換されるかを追跡する。
3	権限判定	権限判定 — 失敗を再現し、型またはログから原因を特定する。
4	パスワードハッシュ	パスワードハッシュ — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

認証と認可とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08 · MINIMUM CODE まず動く最小コード

認証と認可だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Clone, Copy)]
struct UserId(u64);
fn can_edit(user: UserId, owner: UserId) -> bool { user.0 == owner.0 }
fn main() {
    println!("allowed={}", can_edit(UserId(7), UserId(7)));
}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Clone, Copy)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>struct UserId(u64);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn can_edit(user: UserId, owner: UserId) -> bool { user.0 == owner.0 }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>println!("{}", can_edit(UserId(7), UserId(7)));</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が本人確認の条件を満たすか確認する。
- ✓ セッションを実行し、途中の型と状態を追う。
- ✓ 権限判定で失敗を成功経路から分離する。
- ✓ パスワードハッシュにより、出力と副作用を検証する。

型を声に出す 認証と認可に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

allowed=true

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(PartialEq)]
struct UserId(u64);
fn authorize(actor: &UserId, owner: &UserId) -> Result<(), &'static str> {
    if actor == owner { Ok(()) } else { Err("forbidden") }
}
fn main() { assert!(authorize(&UserId(1), &UserId(2)).is_err()); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

各リクエストでセッションを検証し、得たuser_idをSQLのWHERE条件にも含めます。URLのIDだけを信用するとIDOR脆弱性になります。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	本人確認に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	セッションの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	権限判定またはパスワードハッシュの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

認証と認可を再現して切り分ける

認証と認可の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 本人確認
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: セッション
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 権限判定
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: パスワードハッシュ

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	本人確認	本人確認 — 入力と前提を確認し、境界を曖昧にしない。
2	セッション	セッション — 値がどの順で変換されるかを追跡する。
3	権限判定	権限判定 — 失敗を再現し、型またはログから原因を特定する。
4	パスワードハッシュ	パスワードハッシュ — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 認証結果を得た後、対象資源ごとに認可を必ず行います。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

owner_idが異なる場合に更新を拒否する関数とテストを作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(PartialEq)]
struct UserId(u64);

fn authorize(actor: &UserId, owner: &UserId) -> Result<(), &'static str> {
    if actor == owner { Ok(()) } else { Err("forbidden") }
}

fn main() { assert!(authorize(&UserId(1), &UserId(2)).is_err()); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 認証結果を得た後、対象資源ごとに認可を必ず行います。
- ✓ 認証と認可とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

コンパイラエラーとの付き合い方

UNIT 09 · CONCEPT

コンパイラエラーとの付き合い方

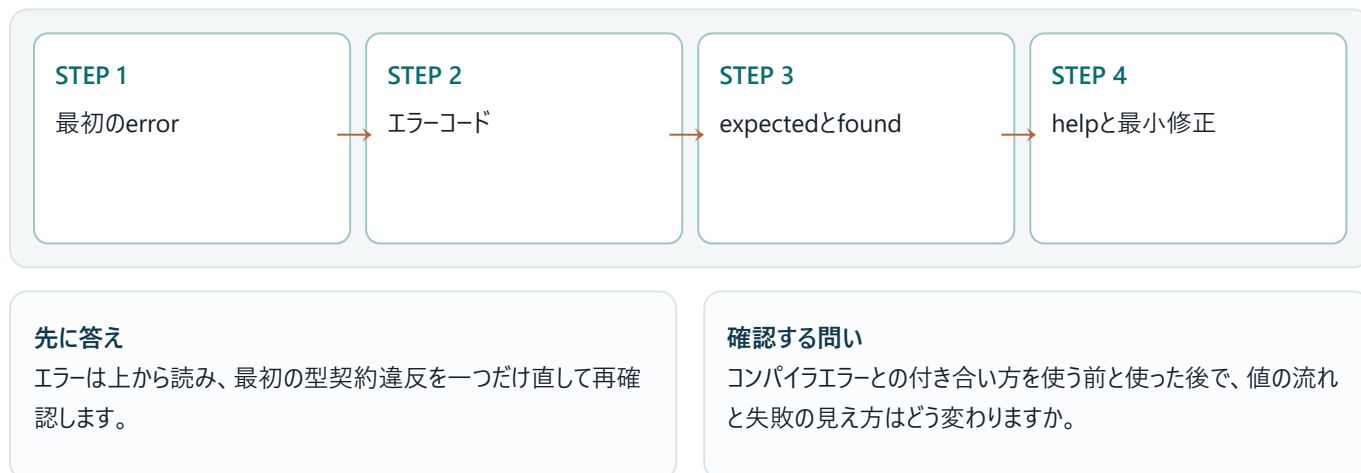
Rustのエラーは失敗宣告ではなく、コンパイラが見つけた契約違反の診断書です。エラーコード、原因行、expected/found、helpを順に読み、最小変更で再コンパイルします。

この単元の到達点

- 最初のerror
- エラーコード
- expectedとfound
- helpと最小修正

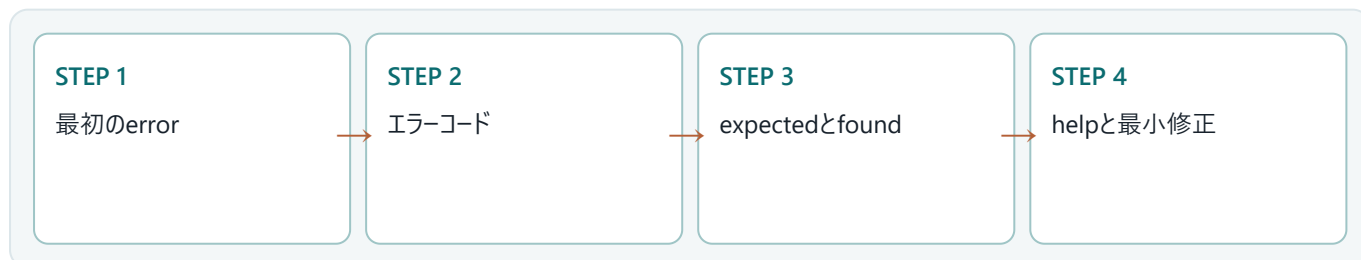
なぜバックエンドで必要か

型エラーを丁寧に直す力はHTTPハンドラ、DB行変換、非同期処理の実装速度へ直結します。派生エラーより最初の原因を優先します。



UNIT 09 · MENTAL MODEL

コンパイラエラーとの付き合い方を図でつかむ



Rustのエラーは失敗宣告ではなく、コンパイラが見つけた契約違反の診断書です。エラーコード、原因行、expected/found、helpを順に読み、最小変更で再コンパイルします。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	最初のerror	最初のerror — 入力と前提を確認し、境界を曖昧にしない。
2	エラーコード	エラーコード — 値がどの順で変換されるかを追跡する。
3	expectedとfound	expectedとfound — 失敗を再現し、型またはログから原因を特定する。
4	helpと最小修正	helpと最小修正 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

コンパイラエラーとの付き合い方とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09 · MINIMUM CODE

まず動く最小コード

コンパイラエラーとの付き合い方だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn main() {  
    let count: u32 = "3";  
    println!("{count}");  
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let count: u32 = "3";</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>println!("{}", count);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が最初のerrorの条件を満たすか確認する。
- ✓ エラーコードを実行し、途中の型と状態を追う。
- ✓ expectedとfoundで失敗を成功経路から分離する。
- ✓ helpと最小修正により、出力と副作用を検証する。

型を声に出す コンパイラエラーとの付き合い方に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
error[E0308]: mismatched types
expected `u32`, found `&str`
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn main() -> Result<(), std::num::ParseIntError> {
    let count: u32 = "3".parse()?;
    println!("{count}");
    Ok(())
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

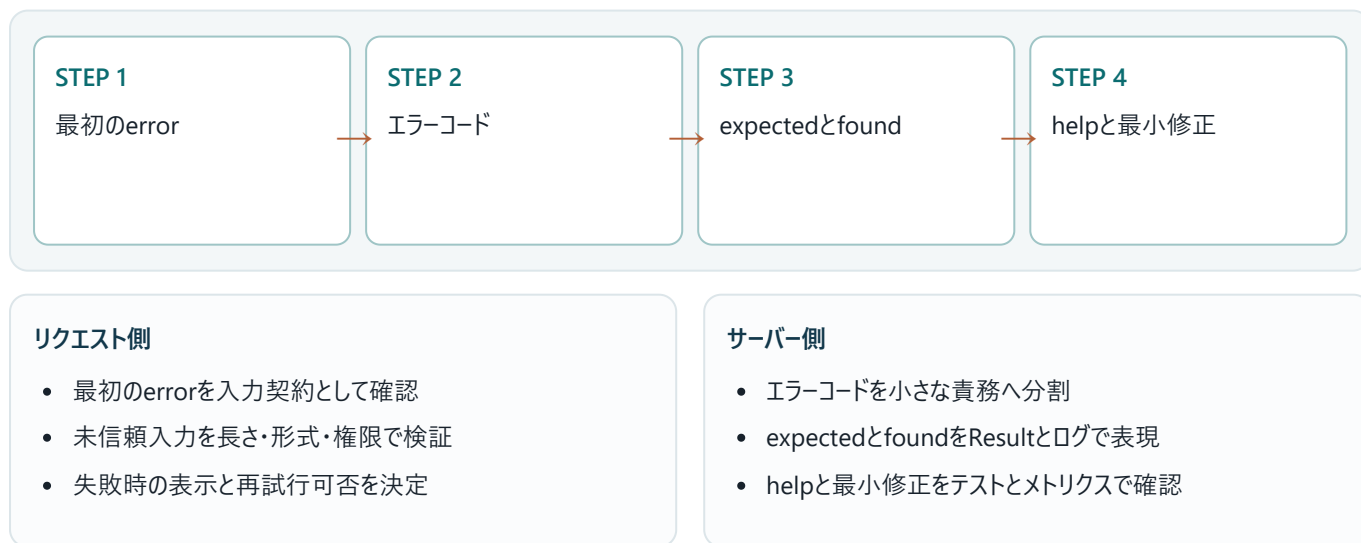
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 · BACKEND APPLICATION

バックエンドのどこで使うか

型エラーを丁寧に直す力はHTTPハンドラ、DB行変換、非同期処理の実装速度へ直結します。派生エラーより最初の原因を優先します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	最初のerrorに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違ふ	エラーコードの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	expectedとfoundまたはhelpと最小修正の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error[E0308]: mismatched types
--> src/main.rs:2:22
2 | let count: u32 = "3";
  |               ---   ^^^ expected `u32`, found `&str`
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

コンパイラエラーとの付き合い方を再現して切り分ける

コンパイラエラーとの付き合い方の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 最初のerror
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: エラーコード
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: expectedとfound
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: helpと最小修正

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	最初のerror	最初のerror — 入力と前提を確認し、境界を曖昧にしない。
2	エラーコード	エラーコード — 値がどの順で変換されるかを追跡する。
3	expectedとfound	expectedとfound — 失敗を再現し、型またはログから原因を特定する。
4	helpと最小修正	helpと最小修正 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 エラーは上から読み、最初の型契約違反を一つだけ直して再確認します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

文字列3をu32へparseし、失敗をResultで返すよう直してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn main() -> Result<(), std::num::ParseIntError> {  
    let count: u32 = "3".parse()?;  
    println!("{count}");  
    Ok(())  
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ エラーは上から読み、最初の型契約違反を一つだけ直して再確認します。
- ✓ コンパイラエラーとの付き合い方とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

学習サイクルと完成像

UNIT 10 · CONCEPT

学習サイクルと完成像

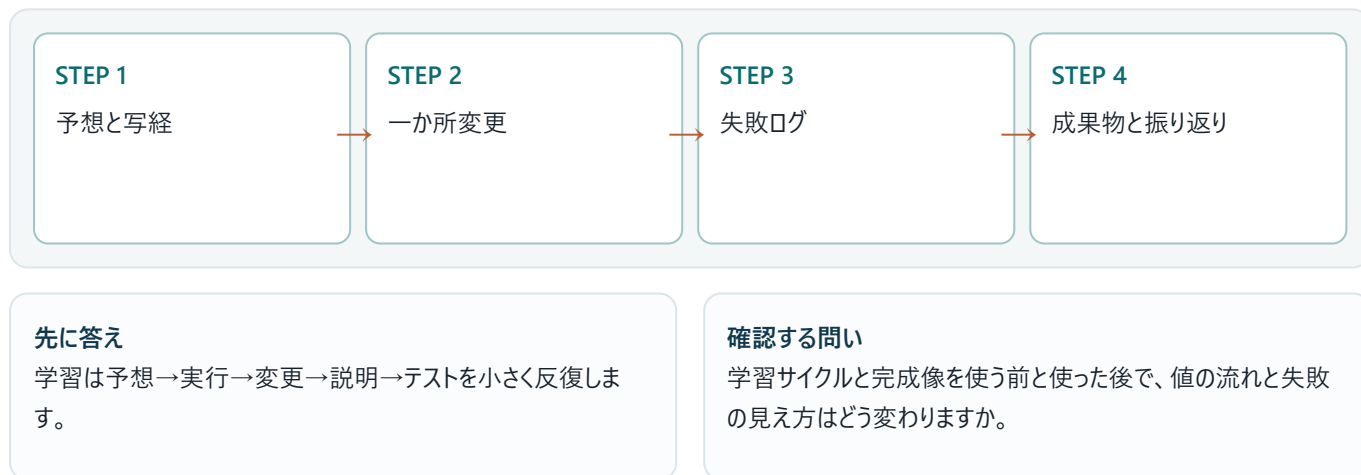
教材を読んだ量ではなく、自分で予想し、実行し、説明し、壊し、直し、テストへ固定できたかで進捗を測ります。最終成果はRustNoteをFly.ioで公開し、認証付きCRUDを運用できることです。

この単元の到達点

- 予想と写経
- 一か所変更
- 失敗ログ
- 成果物と振り返り

なぜバックエンドで必要か

実務では再現手順、テスト、判断記録がコードと同じくらい重要です。学習時から同じ形式で残すと障害対応とレビューへ移行しやすくなります。



UNIT 10 · MENTAL MODEL

学習サイクルと完成像を図でつかむ



教材を読んだ量ではなく、自分で予想し、実行し、説明し、壊し、直し、テストへ固定できたかで進捗を測ります。最終成果はRustNoteをFly.ioで公開し、認証付きCRUDを運用できることです。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では

結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	予想と写経	予想と写経 — 入力と前提を確認し、境界を曖昧にしない。
2	一か所変更	一か所変更 — 値がどの順で変換されるかを追跡する。
3	失敗ログ	失敗ログ — 失敗を再現し、型またはログから原因を特定する。
4	成果物と振り返り	成果物と振り返り — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

学習サイクルと完成像とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

学習サイクルと完成像だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn main() {
    let cycle = ["predict", "run", "change", "explain", "test"];
    for (n, step) in cycle.iter().enumerate() {
        println!("{}", step, n + 1);
    }
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let cycle = ["predict", "run", "change", "explain", "test"];</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>for (n, step) in cycle.iter().enumerate() {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>println!("{}", {step}, n + 1);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が予想と写経の条件を満たすか確認する。
- ✓ 一か所変更を実行し、途中の型と状態を追う。
- ✓ 失敗ログで失敗を成功経路から分離する。
- ✓ 成果物と振り返りにより、出力と副作用を検証する。

型を声に出す 学習サイクルと完成像に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

1. predict
2. run
3. change
4. explain
5. test

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn main() {
    let milestones = [(1, "Rust CLI + tests"), (2, "Topcoat + Turso CRUD"), (3, "Auth + security tests"),
(4, "Fly.io deployment")];
    for (week, result) in milestones { println!("week {week}: {result}"); }
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

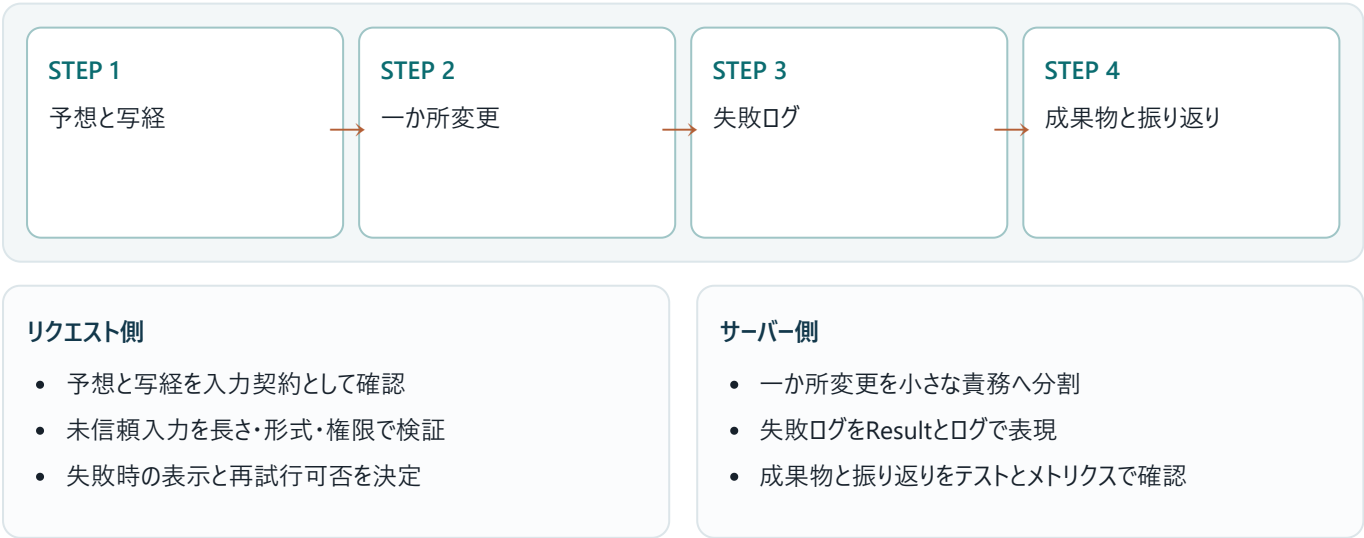
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10・BACKEND APPLICATION

バックエンドのどこで使うか

実務では再現手順、テスト、判断記録がコードと同じくらい重要です。学習時から同じ形式で残すと障害対応とレビューへ移行しやすくなります。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	予想と写経に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	一か所変更の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	失敗ログまたは成果物と振り返りの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

学習サイクルと完成像を再現して切り分ける

学習サイクルと完成像の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 予想と写経
- 2. STEP 2 — 最小入力と同じ症状が出るか確認する。対象: 一か所変更
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 失敗ログ
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 成果物と振り返り

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	予想と写経	予想と写経 — 入力と前提を確認し、境界を曖昧にしない。
2	一か所変更	一か所変更 — 値がどの順で変換されるかを追跡する。
3	失敗ログ	失敗ログ — 失敗を再現し、型またはログから原因を特定する。
4	成果物と振り返り	成果物と振り返り — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 学習は予想→実行→変更→説明→テストを小さく反復します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

14巻を4～8週間で進める計画を作り、各週に動く成果物、確認コマンド、説明できることを一つずつ設定してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn main() {  
    let milestones = [(1, "Rust CLI + tests"), (2, "Topcoat + Turso CRUD"), (3, "Auth + security tests"),  
    (4, "Fly.io deployment")];  
    for (week, result) in milestones { println!("week {week}: {result}"); }  
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 学習は予想→実行→変更→説明→テストを小さく反復します。
- ✓ 学習サイクルと完成像とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

学習用バックエンドの設計図と4週間の実行計画を作る

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	コンピュータ・OS・アプリ	OSはプロセスへ資源を貸し、プログラムは標準入出力と終了コードで外部へ状態を伝えます。
02	ファイル・フォルダ・パス	パスの問題では、まず現在ディレクトリと実在する絶対パスを表示します。
03	端末・シェル・コマンド	コマンドは実行場所、シェル、引数、終了コードの四点をセットで記録します。
04	ソースコードから実行まで	コンパイル時に防げる失敗と、実行時に回復すべき失敗を分けます。
05	ブラウザ・URL・HTTP	Webの一周は、URL→接続→HTTP→サーバー→DB→HTTP→描画です。
06	サーバー・ポート・localhost	接続問題ではhost、port、bind先、実際の待受プロセスを別々に確認します。
07	データベースと永続化	残す必要がある状態は、寿命・一意性・整合性を決めてDBへ保存します。
08	認証と認可	認証結果を得た後、対象資源ごとに認可を必ず行います。
09	コンパイラエラーとの付き合い方	エラーは上から読み、最初の型契約違反を一つだけ直して再確認します。
10	学習サイクルと完成像	学習は予想→実行→変更→説明→テストを小さく反復します。

完了条件

- ✓ 完成画面、入力、保存先、公開先が一枚の図になっている
- ✓ 各週の成果物と確認コマンドが決まっている
- ✓ 一次資料が学習項目へ紐付いている
- ✓ 失敗ログを残す場所と形式が決まっている

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. コンピュータ・OS・アプリを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. ファイル・フォルダ・パスを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. 端末・シェル・コマンドを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. ソースコードから実行までを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. ブラウザ・URL・HTTPを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. サーバ・ポート・localhostを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. データベースと永続化を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. 認証と認可を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. コンパイラエラーとの付き合い方を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. 学習サイクルと完成像を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. コンピュータ・OS・アプリ

コンピュータ・OS・アプリとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: OSはプロセスへ資源を貸し、プログラムは標準入出力と終了コードで外部へ状態を伝えます。

02. ファイル・フォルダ・パス

ファイル・フォルダ・パスとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: パスの問題では、まず現在ディレクトリと実在する絶対パスを表示します。

03. 端末・シェル・コマンド

端末・シェル・コマンドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: コマンドは実行場所、シェル、引数、終了コードの四点をセットで記録します。

04. ソースコードから実行まで

ソースコードから実行までとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: コンパイル時に防げる失敗と、実行時に回復すべき失敗を分けます。

05. ブラウザ・URL・HTTP

ブラウザ・URL・HTTPとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: Webの一周は、URL→接続→HTTP→サーバー→DB→HTTP→描画です。

06. サーバー・ポート・localhost

サーバー・ポート・localhostとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 接続問題ではhost、port、bind先、実際の待受プロセスを別々に確認します。

07. データベースと永続化

データベースと永続化とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 残す必要がある状態は、寿命・一意性・整合性を決めてDBへ保存します。

08. 認証と認可

認証と認可とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 認証結果を得た後、対象資源ごとに認可を必ず行います。

09. コンパイラエラーとの付き合い方

コンパイラエラーとの付き合い方とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: エラーは上から読み、最初の型契約違反を一つだけ直して再確認します。

10. 学習サイクルと完成像

学習サイクルと完成像とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 学習は予想→実行→変更→説明→テストを小さく反復します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
コンピュータ・OS・アプリ	OSはプロセスへ資源を貸し、プログラムは標準入出力と終了コードで外部へ状態を伝えます。	サーバーは長時間動くプロセスです。OSからポート、メモリ、ファイル、環境変数を借り、終了時には接続を安全に閉じます。
ファイル・フォルダ・パス	パスの問題では、まず現在ディレクトリと実在する絶対パスを表示します。	サーバーは設定や静的ファイルを読みます。実行場所に依存する相対パスは開発PCとコンテナで異なるため、基準位置を明示します。
端末・シェル・コマンド	コマンドは実行場所、シェル、引数、終了コードの四点をセットで記録します。	ビルド、テスト、DB操作、デブロイはコマンドから再現できる形で残します。GUIだけの操作は共有と自動化が難しくなります。
ソースコードから実行まで	コンパイル時に防げる失敗と、実行時に回復すべき失敗を分けます。	Rustバックエンドは起動前に多くの型不整合を排除できます。一方、ネットワーク切断やDB障害は実行時の失敗としてResultで扱います。
ブラウザ・URL・HTTP	Webの一周は、URL→接続→HTTP→サーバー→DB→HTTP→描画です。	HTTPサーバーはポートで待ち受け、リクエストを検証し、状態コード・ヘッダー・本文を返します。Topcoatが抽象化しても基礎は障害切り分けに必要です。
サーバー・ポート・localhost	接続問題ではhost、port、bind先、実際の待受プロセスを別々に確認します。	Fly.ioのコンテナでは0.0.0.0とPORT=8080で待ち受けます。127.0.0.1だけにbindするとプロキシから到達できません。
データベースと永続化	残す必要がある状態は、寿命・一意性・整合性を決めてDBへ保存します。	HTTPリクエストごとに作る一時変数と、利用者のノートやセッションのようにDBへ保存する状態を区別します。
認証と認可	認証結果を得た後、対象資源ごとに認可を必ず行います。	各リクエストでセッションを検証し、得たuser_idをSQLのWHERE条件にも含めます。URLのIDだけを信用するとIDOR脆弱性になります。
コンパイラエラーとの付き合い方	エラーは上から読み、最初の型契約違反を一つだけ直して再確認します。	型エラーを丁寧に直す力はHTTPハンドラ、DB行変換、非同期処理の実装速度へ直結します。派生エラーより最初の原因を優先します。
学習サイクルと完成像	学習は予想→実行→変更→説明→テストを小さく反復します。	実務では再現手順、テスト、判断記録がコードと同じくらい重要です。学習時から同じ形式で残すと障害対応とレビューへ移行しやすくなります。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。